Hyperparameter Optimization and Using Learning Curves

Joris Alkema & Ivar van der Spoel

December 2024

1 Introduction

Hyperparameter Optimization (HPO) is essential for enhancing machine learning models but often comes with significant computational costs. Surrogate models address this challenge by predicting model performance using prior data, reducing the need for exhaustive training. This report explores two learning curve-based HPO techniques: Learning Curve Cross-Validation (LCCV) and Inverse Power Law (IPL) extrapolation. Using surrogate models trained on data from the Learning Curve Database (LCDB), these methods aim to estimate model performance efficiently. The goal is to evaluate their predictive accuracy and computational cost.

2 Background

Hyperparameter Optimization (HPO): Tunes hyperparameters to improve machine learning models but is computationally expensive. **Surrogate Models:** Reduce costs by predicting outcomes using historical performance data instead of full training. **LCCV** [MvR23]: Uses learning curves, which show performance over training set sizes, to predict larger-size performance using optimistic extrapolation from anchor points, focusing resources on promising configurations. **IPL** [DSH15]: Fits a power-law function to learning curves to predict performance based on training size, discarding configurations that do not improve on the best-so-far early in the process. **LCDB** [MVLvR22]: Contains empirical learning curve data for various algorithms and datasets, enabling surrogate models to simulate performance effectively and facilitating efficient HPO with methods like LCCV and IPL.

3 Methodology (Experiments)

The efficiency and effectiveness of LCCV and IPL are evaluated through the following experiments: **Implementation Correctness:** The correctness of LCCV and IPL is verified by inspecting whether pruning takes place and if a reasonable score is achieved. **Pruning Efficiency:** Measures the number of configurations pruned early by LCCV and IPL, reflecting their ability to reduce computational costs by focusing on promising candidates. **Pruning Accuracy:** Evaluates the correctness of pruning decisions for configurations pruned before the final anchor by comparing predicted performance with actual performance on the full dataset, assessing the models' reliability in avoiding incorrect pruning. **Anchor strategy:** Anchor size selection might affect how models evaluate the performance. Three different anchor increase strategies were utilized: (1) **Doubling:** Anchor size is multiplied by two per iteration, (2) **Ten Percent:** Anchor size is increased by 10% each iteration, and (3) **Linear:** Anchor size is linearly increased by 1000 each iteration. Please note the default strategy in all other plots is Doubling. It is important to note that due to different strategies, the model has fewer or more iterations before reaching the maximum anchor. These are noted in Table 1.

Added Noise Both models can also perform differently under noisy environments. This was evaluated by adding different levels of Gaussian noise to the performance of the surrogate model (Note: No noise was applied on the final anchor). The noise levels used where $0, 0.1, 0.2$ and 0.5. In some cases, the curve_fit function could not fit the inverse power law in the (standard) 600 iterations. If this happened, a and b were both returned as 1.0.

Strategy	**Anchor steps**
Doubling	10
Ten Percent Increment	73
Linear Increment	16

Table 1: *Average iterations for each strategy, for anchors based on dataset 6*

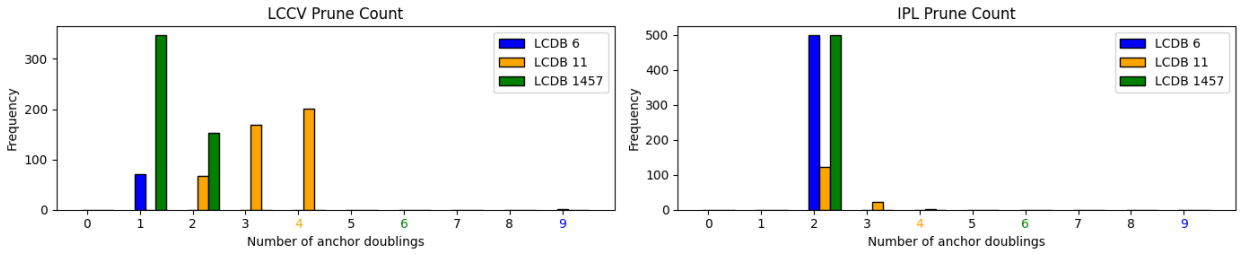

Figure 2: *Pruning frequency of LCCV and IPL on each dataset, where the colored digit represents it's respective last doubling of that specific dataset. Each dataset has minimal anchor* 16*, but final anchors* 16000*,* 500 *and* 1200 *respectively.*

Each experiment is conducted with 500 sampled configurations
and conducted on the 3 datasets supplied by the University. 10 Runs where done per experiment and the results where averaged.

4 Results

This section presents the evaluation of LCCV and IPL across various experiments, focusing on their implementation correctness, pruning efficiency, accuracy, and performance under different anchor strategies and noise conditions. The results highlight the methods' trade-offs in computational efficiency, predictive accuracy, and robustness.

4.1 Implementation Correctness

The goal is to implement two models capable of pruning configurations predicted to perform poorly in the future.

In Figure 1 the output results of LCCV and IPL are presented on LCDB 6. The anchor sizes get doubled every iteration, or are evaluated on the final anchor. On inspection, he implementation works as intended, there is pruning taking place and it is able to find a certain minimal score.

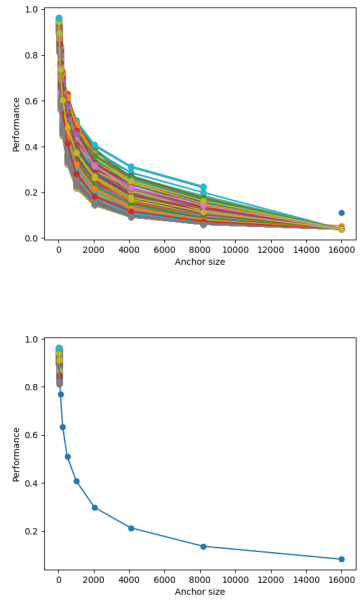

Figure 1: *LCCV and IPL doubling the anchor with 500 configurations on LCDB 6.*

4.2 Pruning Efficiency

However, to show how often pruning takes place, a closer look is taken. By looking at figure 2 the pruning frequency of LCCV and IPL for all supplied datasets can be observed.

The pruning behavior in LCCV varies significantly across different datasets. In the case of LCDB 11, more aggressive pruning occurs after each iteration, with the algorithm becoming increasingly confident that further anchor doubling will not yield a better result. This suggests that the model quickly determines that further exploration would not be worthwhile. On the other hand, LCDB 6 demonstrates minimal pruning, showcasing more optimistic evaluations. The model continues to search for better scores, showing that further exploration might lead to improved results. For LCDB 1457, pruning occurs relatively early, with the model

Dataset	LCCV	IPL
LCDB 6	0/70	209/499
LCDB 11	12/445	0/151
LCDB 1457	165/499	21/499

Table 2: *Pruning accuracy. Denominator refers to the amount of prunes done. Numerator is the amount of prunes done where the final anchor would have been better than the best so far (i.e. wrongfully pruned).*

concluding after one or two anchor doublings that no better performance is likely to be achieved than the initial score found.

In contrast, IPL exhibits a more decisive pruning strategy, where almost all potential solutions are pruned after the first evaluation. Only one solution proceeds to the final anchor, with the rest of the search space being quickly discarded. The exception to this pattern is observed in LCDB 11, where the model remains optimistic about the possibility of finding better performance and continues its search.

4.3 Pruning Accuracy

To assess pruning accuracy, the final anchor was evaluated against the best-so-far configuration for each pruned setup. The results are presented in Table 2. Each model evaluates 500 configurations, with results averaged over 10independent runs. The performance of LCCV and IPL exhibits significant dataset dependency. For example, LCCV achieves perfect pruning accuracy on LCDB 6, wrongfully prunes some configurations on LCDB 11, and exhibits substantial pruning errors on dataset 1457. Notably, both LCCV and IPL prune 499 out of 500 configurations for certain datasets. In such cases, the first configuration, which ends up at the final anchor, is always identified as the best-so-far configuration. Consequently, this approach yields suboptimal accuracy. Ideally, the numerator in all evaluations should be 0, so the accuracy is suboptimal.

4.4 Anchor Strategy, Noise, and Runtime

The performance of LCCV and IPL was evaluated under three anchor size increment strategies and varying levels of Gaussian noise.

LCCV continued to exhibit consistently lower average final performance compared to IPL, indicating its robust utilization of anchor sizes. For both datasets, IPL displayed higher variability, particularly under the ten percent increment strategy. LCCV maintained a steady performance across all strategies, with the linear increment strategy showing a slight degradation on Dataset 11.

Noise robustness experiments revealed that LCCV handled increasing noise levels ($\sigma = 0.1$ and $\sigma = 0.2$) better than IPL, which experienced performance degradation in these ranges. Surprisingly, IPL outperformed LCCV at the highest noise level ($\sigma = 0.5$), potentially due to conservative pruning in extreme conditions. This suggests LCCV is more suitable for low to moderate noise, while IPL can excel under high-noise scenarios.

Runtime Notes: IPL's execution times increased slightly under noisy conditions, while LCCV demonstrated greater consistency in runtime.

5 Discussion

While the datasets are very similar, LCDB 1457 ends with a considerably high score which could cause the improper search for a good configuration. When adding Gaussian noise, IPL took considerably longer to run and often failed to fit the power curve law in less than 600 iterations, it did however perform better then LCCV under the linear anchor strategy , as can be seen in figure 3 and 4 (which had around $6 - 7$ times more iterations than doubling or ten_percent which was interesting.

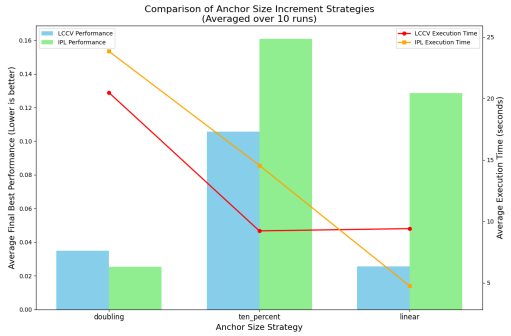

Figure 3: *Dataset 11: Anchor Strategy Comparison. Lower values indicate better performance. Results averaged over 10 runs.*

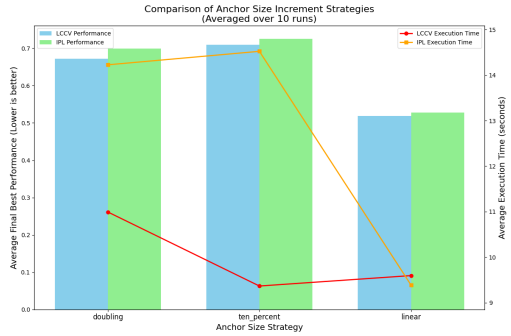

Figure 4: *Dataset 1457: Anchor Strategy Comparison. Lower values indicate better performance. Results averaged over 10 runs.*

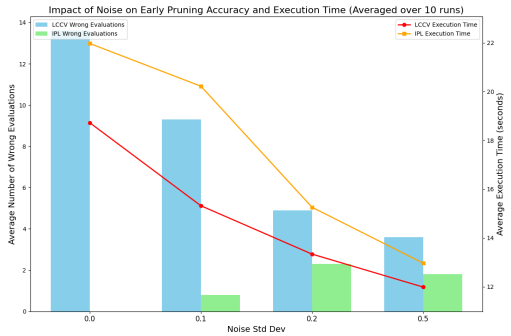

Figure 5: *Dataset 11: Noise Impact Comparison. Lower values indicate better performance. Results averaged over 10 runs.*

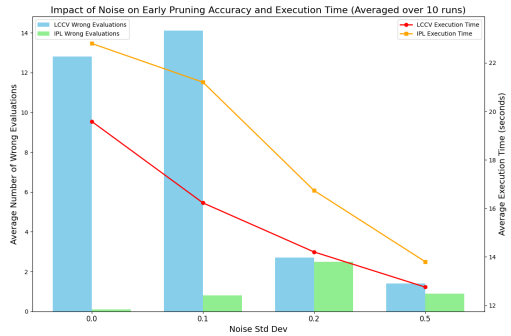

Figure 6: *Dataset 1457: Noise Impact Comparison. Lower values indicate better performance. Results averaged over 10 runs.*

6 Conclusion

This report explored the performance of LCCV and IPL for hyper-parameter optimization, focusing on pruning efficiency, accuracy, and robustness under varying anchor strategies and noise levels. The IPL and LCCV models behave very differently across each dataset. LCCV showed promising extrapolation on LCDB 6 and 11, but just like IPL it was not able to prune properly on LCDB 1457. IPL showed more aggressive pruning and adaptability in high-noise conditions, but at the cost of increased sensitivity to anchor strategy and noise. Overall, LCCV is recommended as the more versatile approach, with IPL offering potential in specific, noise-heavy applications.

References

[DSH15] Tobias Domhan, Jost Tobias Springenberg, and Frank Hutter. Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves. In *Twenty-fourth international joint conference on artificial intelligence*, 2015.

[MVLvR22] Felix Mohr, Tom J Viering, Marco Loog, and Jan N van Rijn. Lcdb 1.0: An extensive learning curves database for classification tasks. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 3–19. Springer, 2022.

[MvR23] Felix Mohr and Jan N van Rijn. Fast and informative model selection using learning curve cross-validation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(8):9669–9680, 2023.